Danmarks Tekniske Universitet

DTU

Manual for EEG device

Version

2025.04.13

April 14, 2025

1 Overview

Contact: lyt225859574@gmail.com
This manual includes the following sections:

- **Overview**: Describes the devices and code structure, including key details and important notes.
- **Usage**: Explains how to connect to the EEG device, save, and visualize data. All instructions are for Windows 10.
- **Development**: Provides guidance on setting up the development environment and modifying parameters.

Please download or clone following Github Repository. Files in "./Codes/" will be used in this manual. Here is the structure:

```
./eegDevice_DTU/Codes —>
    [Arduino] BT_arduino —>
        BT_arduino.py
        classic_bt.ino
    [ESP—IDF] BLE_4—Channel —>
        BLE_visualization.py
        BLE_save_4—channel.py
        esp—idf_code —>
            main
            CMakeLists.txt
            sdkconfig
    [ESP—IDF] BLE_ULP —>
        BLE_visualization.py
        BLE_save_ULP.py
        esp—idf_code —>
            main
            CMakeLists.txt
            sdkconfig
```

There are two EEG devices available in laboratory:

1. **Single Channel EEG Device (SCD)** (fig 1): This is an early prototype. Although it has two channels, only the upper one is in use. It uses AD620 and TL072 and is powered by Âą2.5V to Âą9V. The adjustable potentiometer provides a gain from 7,500 to 4,500 V/V. Since AD620 is less sensitive to electrode quality, this device is more suitable for technical verification.
   As shown in Figure 1, the SCD is powered by two 9V batteries. The output signal ("V_shift") connects to the ADS1115 peripheral, with SDA and SCL lines connected to an ESP32 development board.

Important Notes: The "Ref" pin in the SCD is the reference pin of the AD620 (not the reference electrode) and should be connected to GND. The "5V" and "GND2" pins on the SCD are powered by the 3.3V output from the ESP32 (via USB), used for the DC offset circuit. An internal ADS1115 is also present in the SCD. If you wish to use it, update the following parameters: "ads1115_address = 0x49". By default, in all codes we have "ads1115_address = 0x48".

The lower channel of the SCD uses INA333 and OPA2333, powered by Âą1.5V to Âą2.5V. Do not use a 9V battery with USB together. There is an essential error in the circuit so device is not woring. If you wish to fix it, just replace C38 (11 uF) with a 0.1 uF SMD ceramic capacitor (0805_2012_metric).

2. **Multi-Channel EEG Device (MPD)** (2): The final version of the device, offering 4 channels. It can be powered by a 3.3V Li-Po battery or USB. The main components are INA333 and OPA2333, which are more sensitive to electrode quality.

   If you're getting a consistent 1.5V signal, that is the default dc offset and normal. Try increasing the electrode's contact area with the skin to get EEG signal. Channel 1 includes a DRL (Driven Right Leg) circuit, which may be unstable with poor electrode contact. For initial testing, it's recommended to use **Channel 2**, which doesn't use DRL. By default, when "adc_mode = 1", Channel 2 is selected. To edit it check the Development part.

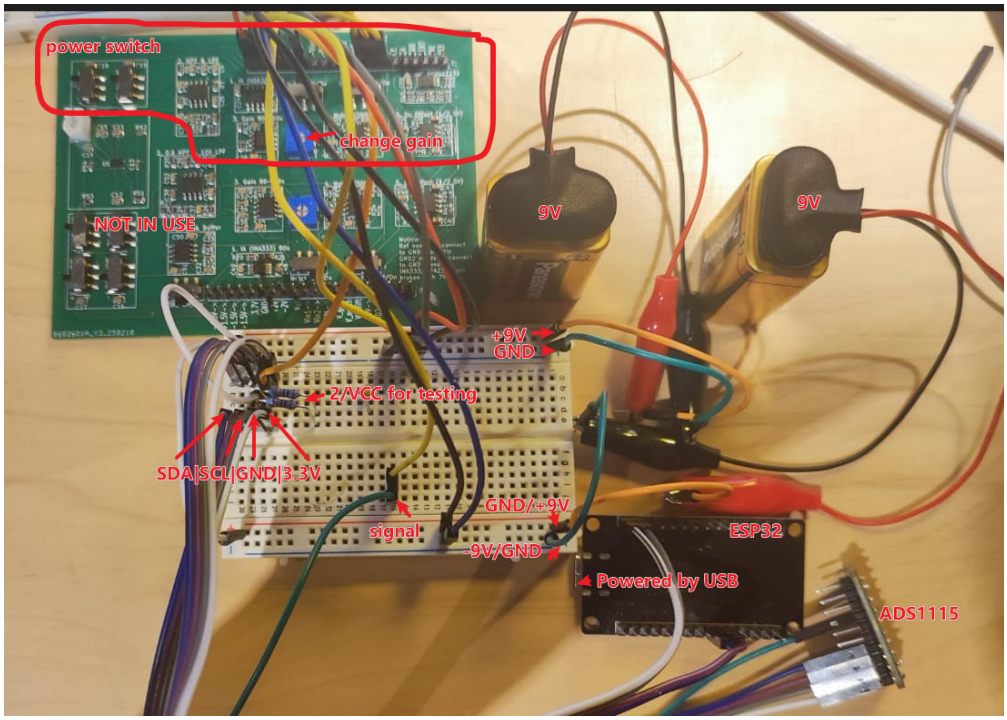

Figure 1: Default setup configure for the SCD. The power are two 9V battery.

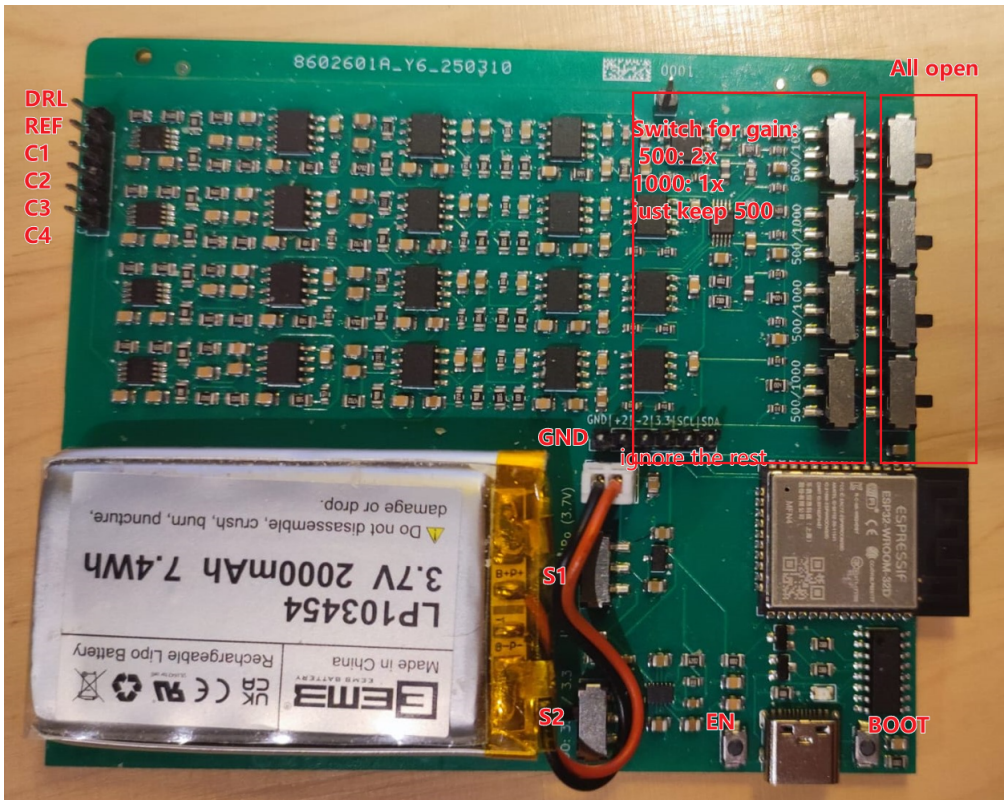

Figure 2: Default setup configure for the SCD. The power is a 3.3V Li-Po battery.

Three software packages are provided for use with the ESP32 and Python:

1. BT_arduino: Classic Bluetooth connection for real-time data visualization. Written in Arduino and recommended for use with the SCD.

2. **BLE_4-Channel**: Written in ESP-IDF. Data is transmitted intermittently via BLE. The "ads1115_mode" parameter in the Python code determines the number of sampled channels. The default mapping is: "1: channel 2", "2: channel 1, channel 2", "3: , channel 1, channel 2, channel 3", "4: channel 1, channel 2, channel 3, channel 4".

3. **BLE_ULP**: Extremely low power mode, written in ESP-IDF. Currently supports only 1 channel. .

Requirements:
For Usage: Python with Bleak, serial, scipy.
For Development: Python, Arduino, ESP-IDF, nRFConnect (mobile), Visual Code Studio or any other IDE.
Before development, be familiar with the config register (Table 9) of the ADS1115 datasheet. The 16-bit config data will be presented as hexadecimal. For example: 1100 0011 1000 0011-> 0XC3 0X83.

2 Usage

2.1 BT_arduino

Steps:

1. Upload classic_bt.ino from the BT_arduino folder to the device (SCD or MCD) using Arduino. By default, SCD was loaded with this code.

2. Ensure the code includes <SerialBT.begin("EEG_Classic_BT");>. This will be the Bluetooth device name.

3. Turn on the device. For SCD, connect two 9V batteries as in Figure 1. Switch on the toggles labeled "+9V" and "-9V" (top left corner). The toggles labeled "notch" and "dc" should remain OFF (they are mislabeled and actually mean ON). Connect signal electrode, reference electrode and GND electrode to Vin1, Vin2 and GND.

4. On your PC, enable Bluetooth function, click "Add Bluetooth or other device", and wait for "EEG_Classic_BT" to appear. Click it to connect.

5. If connected successfully, "EEG_Classic_BT" will show as "paired".

6. Open Device Manager -> "Ports (COM and LPT)" to find the COM port used for Bluetooth.

7. In the BT_arduino folder, open BT_arduino.py in an IDE like Notepad++ or Visual Studio Code.

8. Change the line <PORT = "COM10"> to your actual COM port number. Save and close the script.

9. Open a command prompt, navigate to the folder using <cd F:
eegDevice_DTU
Arduino
BT_arduino>. Or type cmd in the folder address bar and press Enter.

10. Run <python BT_arduino.py>. A window will open with real-time data in three rows.

11. Press "R" to start recording. Data will be saved as EEG_data.txt in the same folder. Press "R" again to stop.

12. Close the window to stop visualization. Once paired, you don't have to scan device again, just connect it through Python code.

2.2 BLE_4-Channel

Steps:

1. Upload the code to the MPD if not already done.
2. Turn on the MPD. Connect the Li-Po battery, then set the "USB(5V)/LiPo(3.7V)" to "LiPo(3.7V)" direction. The LDO switch should remain on "3.3" direction.
3. If you don't know the UUID address of the MPD, continue with the steps below, else jump to step 9.
4. Install the nRFConnector app on your phone.
5. Open the app and tap "Scan".
6. Find and connect to "EEG_BLE" (fig 3).
7. Copy the device address (e.g., 29:43:A8:6E:37:DA) and the UUIDs under "Unknown Service", including data and adc_mode characteristics (fig 4).
8. Open BLE_save_4-channel.py and set DEVICE_ADDRESS, CHARACTERISTIC_UUID, and adc_mode_UUID to your values.
9. Open a command prompt and run <python BLE_save_4-channel.py>. If the device is off or address not right, it will timeout.
10. If connect successful, the data will be saved as EEG_data_yyyymmdd.txt. Multiple recordings on the same day append to the same file.
11. To visualize data, run <python BLE_visualization.py> in another command prompt. Ensure parameters like sampling rate match the code in device.
12. To change the number of channels, edit <adc_mode = 1> in BLE_save_4-channel.py.
13. Connect electrodes to the MPD: For Channel 1: GND electrode, reference electrode and signal electrode are connected to DRL, Ref, C1. For Channels 2 to 4: GND, Ref, C2/C3/C4. By default, adc_mode = 1 uses Channel 2, as it is easier to get data than Channel 1 (which uses DRL).

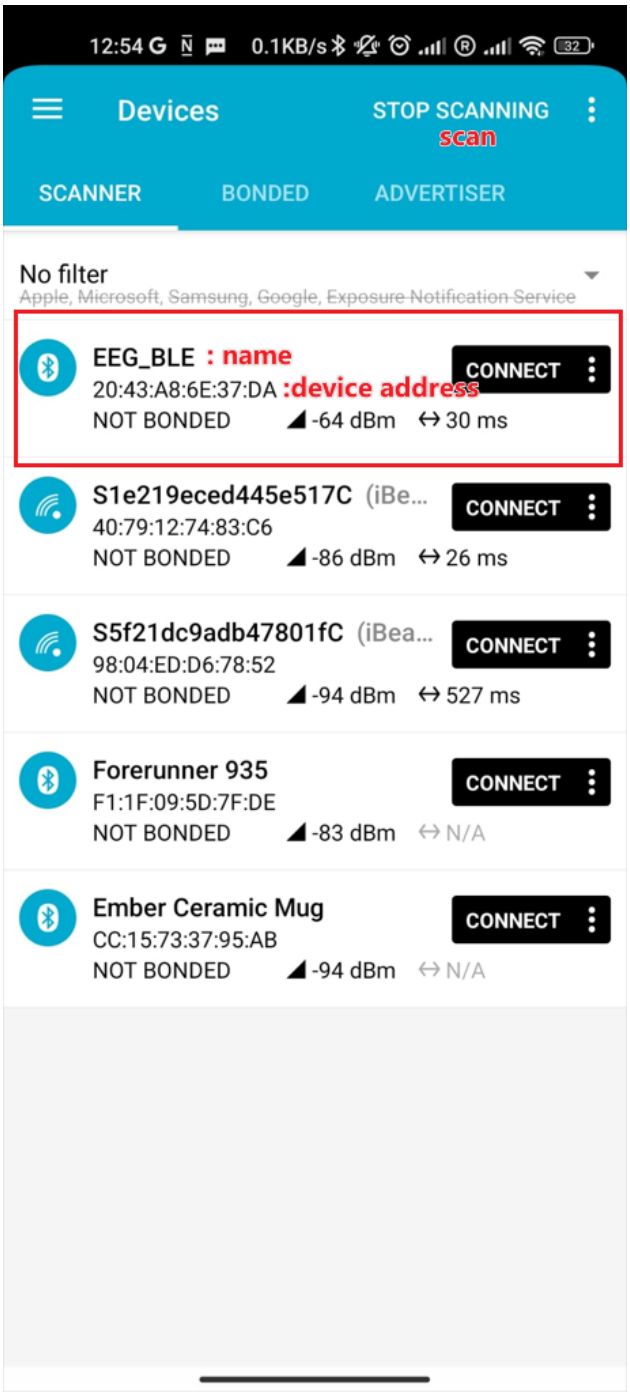

Figure 3: nRFConnector, scan the EEG_device.

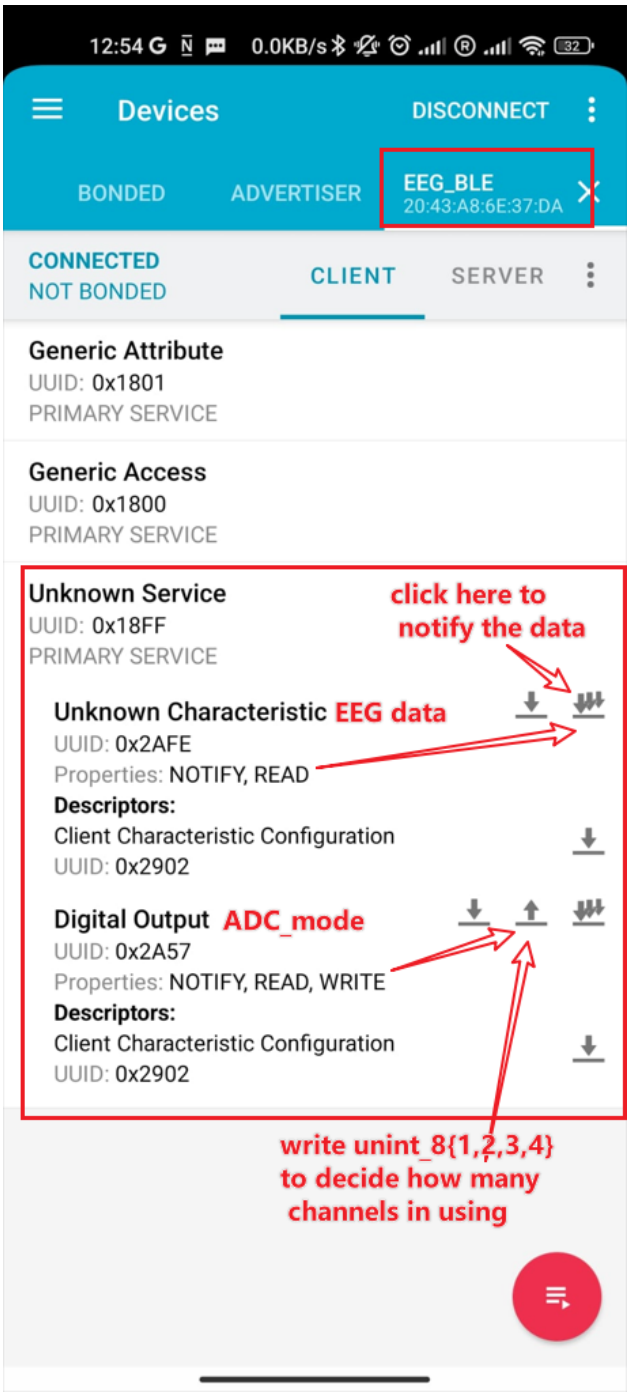

Figure 4: nRFConnector, you can find the UUID for service and characteristics. And using the arrow at right to subscribe notification or write uint8 number to adc_mode directly.

2.3 BLE_ULP

The setup is the same as BLE_4-Channel, with the following differences:
In step 9, run <python BLE_save_ULP.py> instead.

If the device isn't found, the program will automatically restart scanning.

adc_mode is fixed to 1 and should not be changed, as BLE_ULP only supports one channel.

3 Development

3.1 Python code

I didn't add parser in the code, so just change any parameters or edit codes using common IDE like notepad++ or vscode.

You can write your own code, such as searching device by name rather than address, and better searching cycle.

Useful links: Bleak API, scipy.signal.

3.2 Arduino code

3.2.1 Environment Build

1. Install Arduino

2. In "board manager", choose "esp32 by Espressif". Notice, if your User ID contains irregular character like ĂźĂšĂ¤, Chinese or Japanese, choose the old version 2.0.15.
3. In tools -> Board: choose ESP32 Dev Module.
4. In tools ->port: connect the device, usually it will automatically find the COM port when ESP32 is connected to computer using USB.
5. Edit the "classic_bt.ino".
6. After editting, make sure the device is connected in COM port at lower right.
7. Click upload (->).

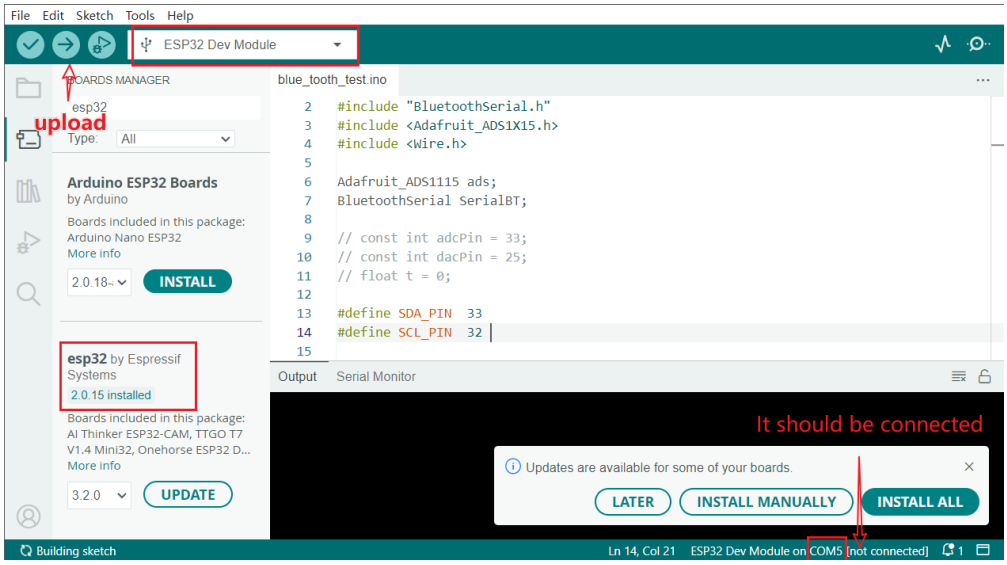

Figure 5: Arduino Editor.

3.2.2 How to code

Useful links: Adafruit_ADS1X15.h

3.2.3 ESP-IDF

3.2.4 Environment build

1. Install ESP-IDF for ESP32 series (v5.4).
2. Install Visual Code Studio. If you are using other IDE just choose the one you are familiar, since we don't need environment from vscode, it is just convenient to edit code.
3. Open esp-idf 5.4 Powershell, run <cd F://eegDevice_DTU//[ESP-IDF] BLE_ULP//esp-idf_code> or BLE_4-channel to move current path with code.
4. run: <idf.py set-target esp32>.

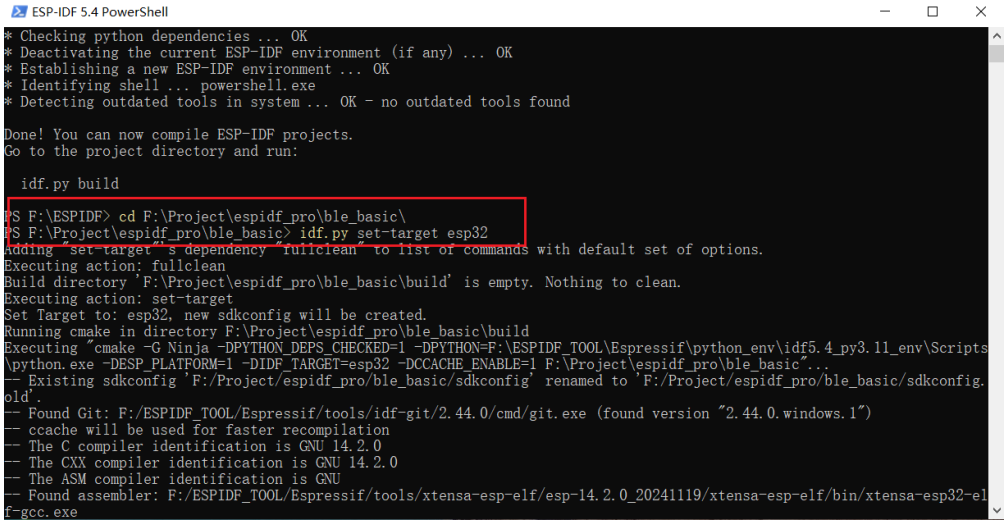

Figure 6: Change path and set target.

5. run: <idf.py menuconfig>. In menuconfig, find:

1) Components config->Bluetooth-> [*] enable
2) Components config->Power Management-> [*] Support for power management
3) Components config->FreeRTOS->Kernal-> [*] configUse_TICKLESS_IDLE
4) Components config->Ultra Low Power co-processor-> [*] enable
5) Components config->Ultra Low Power co-processor->size -> 8176

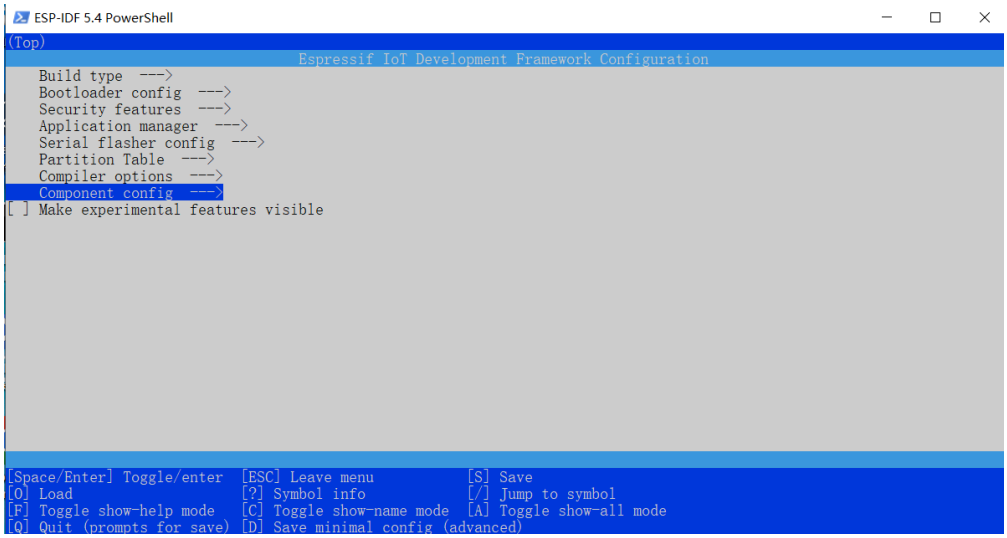

Figure 7: Menu config interface.

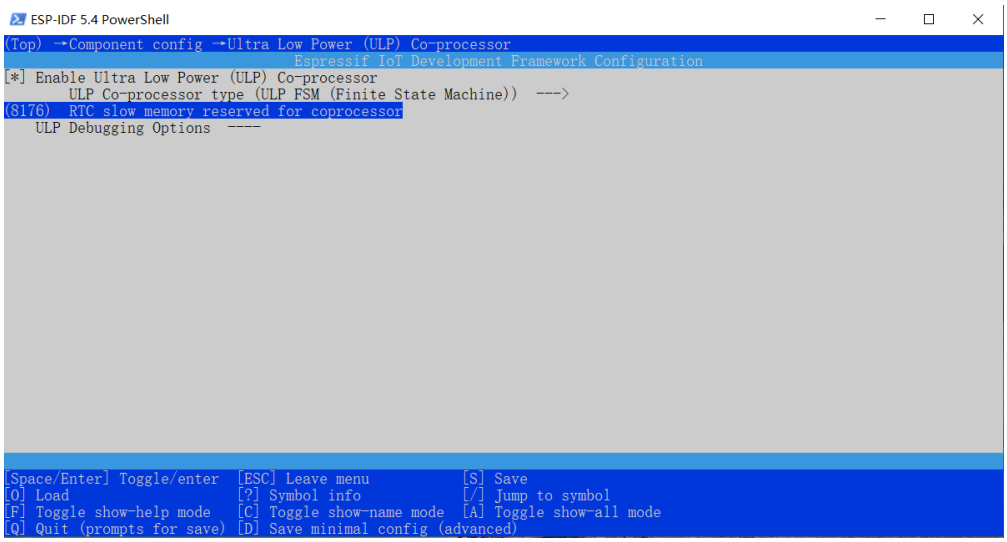

Figure 8: ULP settings.

6. In Visual Code Studio, click File- > Open Folder and choose the esp-idf_code folder.

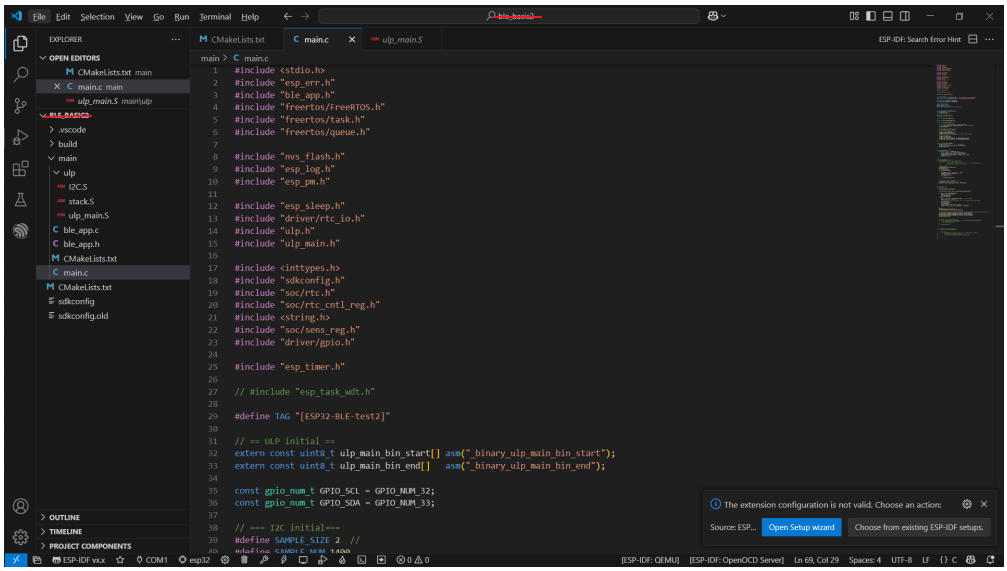

Figure 9: vscode studio interface.

7. Edit Code on you need.

8. After editing, save script and run <idf.py build>. It takes a while for the firt time.

9. Connect device to computer by USB. run <idf.py -p COM? flash>, while COM? is the port connecting device. If you are not sure, open the device manager.

10. After uploading, you can open monitor by runnning <idf.py -p COM? monitor> for debugging or just run Python codes..

3.2.5 How to code

Useful links: ESP-IDF:ULP, ESP-IDF:Sleep, ESP-32: technical manual, Github ULP: examples, Github: BLE example

BLE_4-Channel

In BLE_4-Channel, main.c, you can change the sample rate and interval of sending package. The interval is controlled by <BUFFER_SIZE>. In code below, 5 means 5 seconds, so every 5 seconds, the <eeg_data_buffer> is full and ESP32 will send information to the device.

```
#define SAMPLE_SIZE 2
#define BUFFER_SIZE (5 * SAMPLE_RATE * SAMPLE_SIZE)
uint8_t eeg_data_buffer[BUFFER_SIZE];
```

At the beginning, the adc_mode=0 so device won't start work until there is a connection from computer. In BLE_save_4-channel.py, it will send a message to write the number of "adc_mode" such as 1. Then ESP32 will start to read value and store in main CPU. The memory size of main CPU is larger than in RTC. so you can increase the length of time from 5 second to more.

```
uint8_t ads1115_mode = 0;
```

```
2  ---- in app_main() ----
3      while (ads1115_mode==0)
4      {
5          ESP_LOGI(TAG, "Mode  not setting... (0)");
6          vTaskDelay(pdMS_TO_TICKS(1000));
7      }
8  ---- in ble_app.c: write ads1115_mode ----
9
10 static void gatts_profile_event_handler(esp_gatts_cb_event_t event,
       esp_gatt_if_t gatts_if, esp_ble_gatts_cb_param_t *param)
11 {
12     switch (event) {
13     case ESP_GATTS_WRITE_EVT:
14             if (!param->write.is_prep){
15                 // the data length of gattc write  must be less than
                       GATTS_DEMO_CHAR_VAL_LEN_MAX.
16                 ESP_LOGI(TAG, "GATT_WRITE_EVT, handle = %d, value len = %d,
                       value :", param->write.handle, param->write.len);
17                 esp_log_buffer_hex(TAG, param->write.value, param->write.len
                       );
18                 if(param->write.handle == sv1_handle_table[
                       SV1_CH1_IDX_CHAR_CFG])
19                 {
20                     sv1_ch1_client_cfg[0] = param->write.value[0];
21                     sv1_ch1_client_cfg[1] = param->write.value[1];
22                 }
23                 else if(param->write.handle == sv1_handle_table[
                       SV1_CH2_IDX_CHAR_CFG])
24                 {
25                     sv1_ch2_client_cfg[0] = param->write.value[0];
26                     sv1_ch2_client_cfg[1] = param->write.value[1];
27                 }
28                 else if(param->write.handle == sv1_handle_table[
                       SV1_CH2_IDX_CHAR_VAL])
29                 {
30                     sv1_char2_value[0] = param->write.value[0];
31                     //sv1_char2_value[1] = param->write.value[1];
32                     ads1115_mode = sv1_char2_value[0]; //set ads_mode from
                           computer
33                     // ESP_LOGI(TAG, "channel number: %d", sv1_char2_value
                           [0]);
34                 }
35
36                 if (param->write.need_rsp){
```

```
                    esp_ble_gatts_send_response(gatts_if, param->write.
                        conn_id, param->write.trans_id, ESP_GATT_OK, NULL);
                }
            }
            break;
            ...
            }
```

You can change the behavior of the adc_mode in main.c. There are two parts, in app_main, it will set which channel to convert in ADS1115 and working in continuous mode (for one channel) or single mode (for multiple channel, since it needs to change different channel one by one). In timer_callback, it controls how to read data from ADS1115 in each cycle.

```
--- in app_main() ---
int sample_time = 4000;
    if (ads1115_mode == 1)
    {
        ads1115_config(0xD2, 0xe3);   //  continuous mode, Channel 2
        sample_time = 4000;           // 1/0.004 = 250Hz
    }else if (ads1115_mode == 2)
    {
        ads1115_config(0xC3, 0xe3);   //  single mode, start from channel 1
        sample_time = 5000;           // 1/(2*0.005)=100 Hz
    }else if (ads1115_mode == 3)
    {
        ads1115_config(0xC3, 0xe3);   //  single mode, start from channel 1
        sample_time = 3333;           // 1/(3*0.0033)=100 Hz
    }else if (ads1115_mode == 4)
    {
        ads1115_config(0xC3, 0xe3);   //  danci mode, start from channel 1
        sample_time = 2500;           // 1/(4*0.0025)=100 Hz
    }
--- in timer_callback() ---
nt16_t sample = ads1115_read_adc(); // read data

        if (ads1115_mode != 1) // change channel for next converting
        {
            uint8_t current_channel = (cycle_count % ads1115_mode) + 1;

            switch (current_channel) {
                case 1:
                    ads1115_config(0xc3, 0xe3);
                    break;
                case 2:
```

```
                    ads1115_config(0xd3, 0xe3);
                    break;
                case 3:
                    ads1115_config(0xe3, 0xe3);
                    break;
                case 4:
                    ads1115_config(0xf3, 0xe3);
                    break;
                default:
                    ESP_LOGW("task_sampling", "Invalid channel: %d",
                        current_channel);
                    break;
            }
            cycle_count ++;
        }
```

For the BLE, I do not recommend to edit the profiles. Here I wrote a sending method at the end that can send the data in 500 bytes and automatic al split larger data into 500. You can change it by editing it at the beginning. The largest number is 500 (14 bytes for head information) and decided by MTU.

```
#define MAX_PACKET_SIZE  514  //
```

BLE_ULP

The wake-up interval is decided by sample frequency and number of samples in RTC. In this device, 6 kB is distributed to the data storage, therefore, you can use around 1750*32 = 5.6 kB in RTC_SLOW_MEM, that is 1750 samples. If the sample rate is 250Hz, then the interval is 5 second, and if the sample rate is 100 Hz. the interval is 17 seconds. I recommend the interval being larger than 6 seconds, since each wake up requires full advertising and connecting procedure and requires at least 2.8 seconds. This is mainly limited by the computer to start notification. You can modify the sample frequency and number of samples by follow:

```
--- in main.c ---
#define SAMPLE_SIZE 2  // each data conumpt 16 byte in main CPU, but 32 byte
    in RTC due ULP opereation system
#define SAMPLE_NUM 1750
#define BUFFER_SIZE (SAMPLE_NUM  * SAMPLE_SIZE)

static void init_ulp_program(void)
{
    esp_err_t err = ulp_load_binary(0, ulp_main_bin_start,
            (ulp_main_bin_end - ulp_main_bin_start) / sizeof(uint32_t));
    ESP_ERROR_CHECK(err);

    /* Set ULP wake up period to 1s */
```

```
    ulp_set_wakeup_period(0, 10*1000); // sample rate = 1/0.01=100 Hz

    rtc_gpio_isolate(GPIO_NUM_12);
    rtc_gpio_isolate(GPIO_NUM_15);
    esp_deep_sleep_disable_rom_logging(); // suppress boot messages

    rtc_gpio_init(GPIO_SCL);
    rtc_gpio_init(GPIO_SDA);
    rtc_gpio_set_direction(GPIO_SCL, RTC_GPIO_MODE_INPUT_ONLY);
    rtc_gpio_set_direction(GPIO_SDA, RTC_GPIO_MODE_INPUT_ONLY);
}

--- ulp_main.S ---
define N_SAMPLE      1750
```

To change the covnerting channel of ADS1115 with ULP, you can edit the value of I2C writing:

```
.text
.global entry
entry:
    move r1, doread
    ld r0, r1, 0
    jumpr readvalue, 1, ge

    // write config
    move r3, stackEnd
    psr
    jump i2c_start_cond

    move r2, 0x90
    move r1, 0x00
    psr
    jump i2c_write_byte
    jumpr noack, 1, ge

    move r2, 0x01
    psr
    jump i2c_write_byte
    jumpr noack, 1, ge

    // ADS1115 config write 15-8
    move r2, 0xD2
    psr
    jump i2c_write_byte
```

```
28      jumpr noack, 1, ge
29
30      // ADS1115 config write 7-0
31      move r2, 0xe3
32      psr
33      jump i2c_write_byte
34      jumpr noack, 1, ge
35
36      psr
37      jump i2c_stop_cond
38
39      wait 100000
```